“SISTEMA DE GESTIÓN DE INVENTARIO JORGE MOTORS”

Plataforma web basada en arquitectura en capas, patrones de diseño GoF y modelo C4

AUTOR(ES):
CASTILLO CANALES, KORAYMA ANAÍS
HERNÁNDEZ HERNÁNDEZ, JOSÉ ANTONIO
NEYRA QUIROZ, JOSÉ GUILLERMO
ROJAS CASTILLO, SEBASTIÁN JAREN

DOCENTE:
MG. LUIS ENRIQUE RAMÍREZ PACHECO

CURSO:
ARQUITECTURA DE SOFTWARE – VI – MAÑANA – A1

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

Contenido de la exposición

1–2 Introducción y objetivos

3–4 Descripción y requerimientos

5 Arquitectura y patrones arquitectónicos

6 Patrones GoF en el código

7 Modelo C4, despliegue y decisiones

8 Base de datos

9 Demostración del sistema

10–12 Resultados, conclusiones y bibliografía

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

1. Introducción

El taller

Jorge Motors es una mecánica automotriz de Pueblo Nuevo, Chincha (Ica), que gestiona repuestos, lubricantes, filtros, baterías, herramientas y accesorios.

Problema actual: Excel

- Registros con errores o desactualizados.
- El stock no se actualiza tras cada movimiento.
- Sin alertas cuando un producto llega a su mínimo.

Aplicación web por rol

React 19 con vistas separadas para Administrador y Mecánico.

API REST en capas

Node.js + Express 5 como monolito modular, con 6 patrones GoF en el código.

SQL Server / Azure SQL

12 tablas, 8 migraciones y triggers que protegen el historial.

Entra ID + pruebas

Sin contraseñas locales; 232 pruebas automatizadas aprobadas.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

2. Objetivos

Objetivo general

Diseñar, implementar y verificar una plataforma web segura para la gestión de inventario de "Jorge Motors", aplicando arquitectura en capas, patrones GoF y el modelo C4, que automatice entradas y salidas, garantice la consistencia del stock, conserve un historial auditable y genere alertas de reabastecimiento.

1. Analizar
requerimientos funcionales y no funcionales del taller

2. Diseñar
el monolito modular en capas y modelarlo con C4

3. Implementar
los seis patrones GoF y los complementarios, con pruebas

4. Construir
la base en SQL Server con migraciones versionadas

5. Desarrollar
la interfaz por rol conectada a la API con Entra ID

6. Verificar
consistencia, seguridad y trazabilidad con pruebas

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

3. Descripción del sistema

Actores del sistema

Administrador
Catálogo, proveedores, compras, entradas y salidas, reportes e historial completo.

Mecánico
Consulta el stock sin precios, registra salidas y ve solo su propio historial.

Proveedor de repuestos
Contacto externo; recibe los avisos de reposición y no inicia sesión.

Microsoft Entra ID
Autentica a los empleados y emite el token con su rol de aplicación.

Alcance

Dentro del alcance
- Autenticación y roles
- Categorías, productos y proveedores
- Entradas y salidas con historial inmutable
- Facturas de compra y adjuntos privados
- Alertas de stock mínimo con aviso recuperable
- Reporte PDF / Excel y auditoría

Fuera del alcance
- Contabilidad, facturación a clientes, ventas y pagos
- Diagnóstico de vehículos y órdenes de trabajo completas
- Portal de proveedores

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

3. Descripción del sistema: módulos implementados

Inventario
Entradas y salidas con bloqueo por producto e idempotencia

Catálogo
Categorías y productos con edición versionada (ETag)

Proveedores
Contactos con código único, activos o inactivos

Facturas de compra
Borrador, anulación y recepción que suma el stock

Adjuntos
PDF, PNG o JPEG privados por factura, máximo 5 MiB

Alertas
Alerta al llegar al mínimo y tarea de aviso en el Outbox

Reportes
Inventario con filtros y exportación PDF / Excel

Auditoría
Éxitos y rechazos con usuario y request ID

16	16	12	232
RF	RNF	tablas	pruebas

Estado: backend, base de datos e interfaz implementados y verificados en local con SQL Server 2019. Docker, GitHub Actions y la conexión a Azure SQL quedan preparados; el correo real, Blob Storage y el kardex exportable están pendientes.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

4. Requerimientos funcionales

ID	Requerimiento	Estado	ID	Requerimiento	Estado
RF-01	Inicio de sesión con Entra ID	Implementado	RF-09	Stock consistente (sin negativos ni duplicados)	Implementado
RF-02	Diferenciación de roles	Implementado	RF-10	Trazabilidad de movimientos	Implementado
RF-03	Restricción de acciones por rol	Implementado	RF-11	Reporte de inventario PDF / Excel	Implementado
RF-04	Consulta del usuario autenticado	Implementado	RF-12	Alertas de bajo stock	Implementado
RF-05	Gestión de productos	Implementado	RF-13	Notificación al proveedor	Parcial: correo real pendiente
RF-06	Gestión de categorías	Implementado	RF-14	Facturas de compra	Implementado
RF-07	Gestión de proveedores	Implementado	RF-15	Adjuntos de factura	Implementado (local)
RF-08	Registro de entradas y salidas	Implementado	RF-16	Auditoría de operaciones	Implementado

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

4. Requerimientos no funcionales

ID	Requerimiento	Atributo	ID	Requerimiento	Atributo
RNF-01	Token Entra ID válido en toda ruta	Seguridad	RNF-09	Búsqueda ≤ 2 s (p95)	Rendimiento
RNF-02	Rol del token = perfil activo	Seguridad	RNF-10	Health y logs con request ID	Disponibilidad
RNF-03	Sin contraseñas locales	Seguridad	RNF-11	Rollback total y reintentos finitos	Fiabilidad
RNF-04	Stock en una transacción, nunca negativo	Integridad	RNF-12	Worker independiente de la API	Escalabilidad
RNF-05	Validación estricta con Zod	Integridad	RNF-13	Salida en pocos pasos	Usabilidad
RNF-06	Historial y auditoría inmutables	Auditoría	RNF-14	HTTPS, CORS, Helmet, 64 KB	Seguridad
RNF-07	Capas y módulos por dominio	Mantenibilidad	RNF-15	Idempotencia de movimientos	Integridad
RNF-08	Errores centralizados	Mantenibilidad	RNF-16	ETag / If-Match (412)	Integridad

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

5. Arquitectura: del diseño a la implementación

Aspecto	Avance 2 (diseño)	Implementación actual	Motivo
Acceso a datos	Prisma ORM	node-mssql + T-SQL explícito	Control de transacciones, bloqueos e identidad administrada
Base de datos	SQLite / PostgreSQL	SQL Server 2019 → Azure SQL	Destino acordado en Azure
Autenticación	Correo, contraseña y JWT propio	Microsoft Entra ID (MSAL + PKCE)	No guardar contraseñas; roles y MFA
Avisos	Correo, SMS o WhatsApp	Outbox + worker (correo pendiente)	No perder avisos ni bloquear movimientos
Strategy	Canal de aviso	Formato del reporte PDF / Excel	Alternativa real implementada en el código
Alcance	7 tablas	12 tablas: + compras, adjuntos, auditoría, Outbox	Recepción de compras y trazabilidad
Estilo	Capas + monolito modular	Se mantiene	—

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

5. Arquitectura en capas

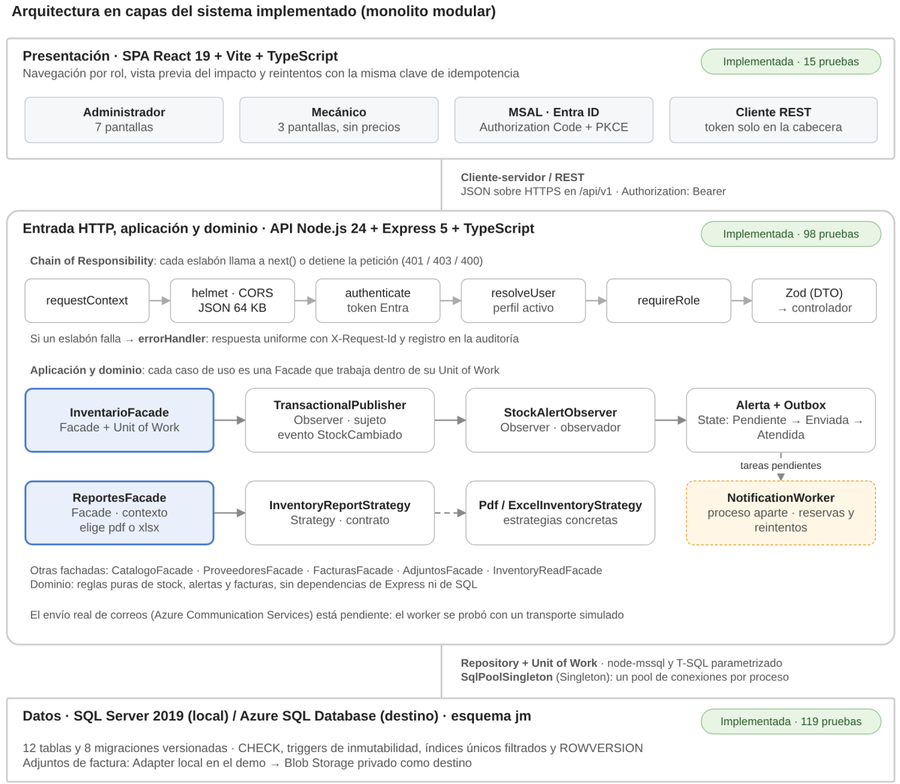


Figura 1. Arquitectura en capas y ubicación de cada patrón

Capas

- **Presentación:** React 19, vistas por rol y MSAL.
- **Entrada HTTP:** rutas y middlewares (Chain of Responsibility).
- **Aplicación:** Facades dentro de su Unit of Work.
- **Dominio:** reglas puras de stock, alertas y facturas.
- **Infraestructura:** repositorios SQL y almacenamiento.

Justificación

- Dependencia en un solo sentido: un cambio queda acotado a su capa.
- El dominio se prueba sin levantar la interfaz ni Express.
- Monolito modular: un solo despliegue, módulos con responsabilidades claras.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

5. Patrones arquitectónicos

ID	Patrón	Aplicación en el sistema
PA-01	Arquitectura en capas	Presentación, entrada HTTP, aplicación, dominio e infraestructura, con dependencias en un solo sentido
PA-02	Monolito modular	Módulos por dominio en backend/src/modules/; el worker reutiliza el mismo código
PA-03	Cliente-servidor / REST	La SPA solo consume /api/v1; nunca accede a SQL ni tiene credenciales de servicios
PA-04	Repository + Unit of Work	Casos de uso sobre contratos; la unidad de trabajo abre la transacción y entrega los repositorios
PA-05	Transactional Outbox	Alerta y tarea de aviso en la misma transacción; un worker las procesa con reservas y reintentos

No se declara **Pipe and Filter**: repetir la cadena HTTP no es una implementación distinta. **Adapter** se aplica en el almacenamiento de adjuntos y el transporte de correo.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

6. Patrones de diseño GoF implementados

PG-01 · Singleton

Creacional

Un pool de conexiones SQL por proceso

SqlPoolSingleton

PG-02 · Facade

Estructural

Un movimiento coordina validación, stock, historial y auditoría

InventarioFacade

PG-03 · Observer

Comportamiento

Alertas desacopladas del inventario

TransactionalPublisher · StockAlertObserver

PG-04 · State

Comportamiento

Transiciones válidas de la alerta

Alerta · Pendiente · Enviada · Atendida

PG-05 · Strategy

Comportamiento

Exportar el reporte en distintos formatos

PdfInventoryStrategy · ExcelInventoryStrategy

PG-06 · Chain of Responsibility

Comportamiento

Reglas HTTP en orden que pueden detener la petición

authenticate → resolveUser → requireRole

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

6. Patrones GoF: Singleton

```ts
export class SqlPoolSingleton {
  private static readonly instance = new SqlPoolSingleton();
  private connection: Promise<SqlConnection> | undefined;
  private settingsKey: string | undefined;
  private constructor() {}

  static getInstance(): SqlPoolSingleton { return this.instance; }

  getConnection(settings: SqlSettings, connector: Connector = connectSql): Promise<SqlConnection> {
    const key = JSON.stringify(settings);
    if (this.connection && this.settingsKey !== key) {
      throw new Error('El pool del proceso ya está asociado a otra configuración.');
    }
    if (!this.connection) {
      this.settingsKey = key;
      this.connection = connector(settings).catch((error: unknown) => {
        this.connection = undefined;
        this.settingsKey = undefined;
        throw error;
      });
    }
    return this.connection;
  }

  async close(): Promise<void> {
    const pending = this.connection;
    if (!pending) return;
    try { await (await pending).pool.close(); }
    finally { this.connection = undefined; this.settingsKey = undefined; }
  }
}
```

Figura 2. sql-pool.ts · SqlPoolSingleton

Problema

Crear un pool de conexiones en cada operación desperdicia conexiones y puede saturar SQL Server.

Solución

Constructor privado e instancia estática con getInstance(). getConnection() comparte la misma promesa; ante un fallo se limpia para reintentar y close() cierra el pool al apagar.

Evidencia

Pruebas: misma instancia y promesa, una sola conexión inicial y reintento tras fallo.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

6. Patrones GoF: Facade

```
export class InventarioFacade {
  constructor(
    private readonly unitOfWork: InventoryUnitOfWork,
    private readonly events: TransactionalPublisher<StockCambiado, Repositories>,
  ) {}

  registrarSalida(input: MovimientoInput, usuarioId: number, requestId = randomUUID()): Promise<ResultadoMovimiento> {
    return this.registrar(input, usuarioId, 'SALIDA', requestId);
  }

  registrarEntrada(input: MovimientoInput, usuarioId: number, requestId = randomUUID()): Promise<ResultadoMovimiento> {
    return this.registrar(input, usuarioId, 'ENTRADA', requestId);
  }

  private async registrar(input: MovimientoInput, usuarioId: number, tipo: TipoMovimiento, requestId: string):
Promise<ResultadoMovimiento> {
    const parsed = movimientoInputSchema.safeParse(input);
    if (!parsed.success || !z.uuid().safeParse(requestId).success || !Number.isInteger(usuarioId) || usuarioId <= 0 || usuarioId >
MAX_STOCK) {
      throw new InventoryError('INVALID_INPUT', 'Los datos del movimiento no son válidos.');
    }
    return this.unitOfWork.ejecutar(async (repositories) => {
      const usuario = await repositories.usuarios.obtener(usuarioId);
      if (!usuario?.activo) throw new InventoryError('USER_NOT_ACTIVE', 'El usuario no está habilitado.');
      return registrarMovimientoEnTransaccion(parsed.data, usuario, tipo, repositories, this.events, requestId);
    });
  }
}
```

Figura 3. inventario.facade.ts · InventarioFacade

Problema

Un movimiento coordina validación, permisos, idempotencia, stock, historial, alertas y auditoría.

Solución

registrarSalida() y registrarEntrada() validan con Zod y ejecutan todo el caso de uso dentro de unitOfWork.ejecutar(), en una sola transacción. El mismo enfoque usan Catálogo, Proveedores, Facturas, Adjuntos y Reportes.

Evidencia

Dos salidas simultáneas sobre stock 3 confirman solo una; un fallo en el historial revierte el descuento.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

6. Patrones GoF: Observer

```ts
export interface TransactionalObserver<Event, Context> {
  actualizar(event: Readonly<Event>, context: Context): Promise<void>;
}

// Observer interno: se espera a cada suscriptor ANTES del commit de la UoW.
// No es un broker durable; la durabilidad depende de los repositorios Outbox.
export class TransactionalPublisher<Event, Context> {
  private readonly observers = new Set<TransactionalObserver<Event, Context>>();

  suscribir(observer: TransactionalObserver<Event, Context>): () => void {
    this.observers.add(observer);
    return () => { this.observers.delete(observer); };
  }

  async publicar(event: Readonly<Event>, context: Context): Promise<void> {
    for (const observer of [...this.observers]) await observer.actualizar(event, context);
  }
}

```

Figura 4. transactional-publisher.ts · sujeto

```ts
import { Alerta, requiereAlerta } from '../domain/alerta.js';
import type { StockCambiado } from './contracts.js';
import type { Repositories } from '../../inventario/application/contracts.js';
import type { TransactionalObserver } from '../../../shared/transactional-publisher.js';

export class StockAlertObserver implements TransactionalObserver<StockCambiado, Repositories> {
  async actualizar(event: Readonly<StockCambiado>, repositories: Repositories): Promise<void> {
    const { producto, stockActual, usuario, requestId } = event;
    const abierta = await repositories.alertas.buscarAbierta(producto.id);
    const bajo = requiereAlerta(stockActual, producto.stockMinimo);
    if (bajo && !abierta) {
      const alerta = await repositories.alertas.crear(event);
      // Outbox: pendiente durable en la misma transación que movimiento y alerta.
      // Nunca se llama a SMTP/ACS ni a servicios externos desde este suscriptor.
      await repositories.outbox.encolar(alerta.id, event);
      await repositories.auditoria.registrar({
        requestId, action: 'ALERTA_CREADA', result: 'EXITO', userId: usuario.id,
        tenantId: usuario.entraTenantId, objectId: usuario.entraObjectId,
        entityId: String(alerta.id), code: 'LOW_STOCK_ALERT_CREATED', httpStatus: event.httpStatus,
        before: null, after: { estado: alerta.estado, stock: stockActual },
      });
    } else if (!bajo && abierta) {
      const alerta = new Alerta(abierta.estado);
      alerta.atender();
      await repositories.alertas.atender(abierta.id, abierta.estado);
      await repositories.outbox.cancelarNoAceptada(abierta.id);
      await repositories.auditoria.registrar({
        requestId, action: 'ALERTA_ATENDIDA', result: 'EXITO', userId: usuario.id,
        tenantId: usuario.entraTenantId, objectId: usuario.entraObjectId,
        entityId: String(abierta.id), code: event.movimientoId === null ? 'CATALOG_THRESHOLD_RESOLVED' : 'STOCK_REPLENISHED', httpStatus:
  event.httpStatus,
        before: { estado: abierta.estado }, after: { estado: alerta.estado, stock: stockActual },
      });
    }
  }
}

```

Figura 5. stock-alert.observer.ts · observador

Problema

Inventario no debe conocer al módulo de alertas para crear o atender una alerta.

Solución

TransactionalPublisher (sujeto) suscribe observadores y publica StockCambiado. StockAlertObserver (observador) crea la alerta + Outbox o atiende la abierta. Se ejecuta antes del commit, en la misma transacción.

Evidencia

Umbral inclusivo, sin alertas duplicadas, atención al reponer y rollback si falla el Outbox.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

6. Patrones GoF: State

```typescript

interface EstadoAlerta {
  readonly nombre: NombreEstadoAlerta;
  notificacionAceptada(): EstadoAlerta;
  atender(): EstadoAlerta;
}

class Pendiente implements EstadoAlerta {
  readonly nombre = 'PENDIENTE';
  notificacionAceptada(): EstadoAlerta { return new Enviada(); }
  atender(): EstadoAlerta { return new Atendida(); }
}
class Enviada implements EstadoAlerta {
  readonly nombre = 'ENVIADA';
  notificacionAceptada(): EstadoAlerta { throw new Error('La alerta ya tiene una notificación aceptada.'); }
  atender(): EstadoAlerta { return new Atendida(); }
}
class Atendida implements EstadoAlerta {
  readonly nombre = 'ATENDIDA';
  notificacionAceptada(): EstadoAlerta { throw new Error('Una alerta atendida no puede volver a enviarse.'); }
  atender(): EstadoAlerta { throw new Error('La alerta ya fue atendida.'); }
}

// State GoF: cada objeto de estado decide qué comportamiento/transición admite.
export class Alerta {
  private state: EstadoAlerta;
  constructor(nombre: NombreEstadoAlerta = 'PENDIENTE') {
    switch (nombre) {
      case 'PENDIENTE': this.state = new Pendiente(); break;
      case 'ENVIADA': this.state = new Enviada(); break;
      case 'ATENDIDA': this.state = new Atendida(); break;
      default: throw new Error('Estado de alerta desconocido.');
    }
  }
  get estado(): NombreEstadoAlerta { return this.state.nombre; }
  notificacionAceptada(): void { this.state = this.state.notificacionAceptada(); }
  atender(): void { this.state = this.state.atender(); }
}
```

Figura 6. alerta.ts · Pendiente → Enviada → Atendida

Problema

Cada estado de la alerta admite acciones distintas: una alerta atendida no puede volver a enviarse.

Solución

Interfaz EstadoAlerta con notificacionAceptada() y atender(). Las clases Pendiente, Enviada y Atendida devuelven el siguiente estado o rechazan la transición. El contexto Alerta delega en su estado.

Evidencia

Pruebas de transiciones válidas e inválidas; CHECK en la base como segunda barrera.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

6. Patrones GoF: Strategy

```ts
// Strategy: la aplicación conoce este contrato, no PDFKit ni ExcelJS.
export interface InventoryReportStrategy {
  generar(report: InventoryReport, generadoEn: string): Promise<Buffer>;
}
```

Figura 7. inventory-report.ts · contrato



Figura 8. report-strategies.ts · ExcelInventoryStrategy

Problema

Exportar el reporte en PDF y Excel sin condicionales repartidos ni dependencia de las bibliotecas.

Solución

Contrato InventoryReportStrategy.generar(). Estrategias concretas Pdf y Excel. ReportesFacade.exportarInventario() es el contexto: valida, lee los datos, elige la estrategia por formato y audita.

Evidencia

El mismo caso de uso funciona con las dos estrategias reales y con estrategias simuladas.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

6. Patrones GoF: Chain of Responsibility

```ts
app.use(requestContext(logger));
app.use((req, res, next) => {
  res.locals.auditRequired = req.path.startsWith('/api/v1/inventario') || req.path.startsWith('/api/v1/catalogo') ||
req.path.startsWith('/api/v1/compras') || req.path.startsWith('/api/v1/reportes') || req.path === '/api/v1/auth/me';
  next();
});
app.use(helmet());
app.use(cors({
  origin: (origin, callback) => {
    if (!origin || origin === config.FRONTEND_ORIGIN) {
      callback(null, true);
      return;
    }
    callback(new AppError(403, 'ORIGIN_NOT_ALLOWED', 'El origen no está permitido.'));
  },
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
  exposedHeaders: ['X-Request-Id', 'ETag', 'Content-Disposition', 'Retry-After'],
}));
app.use(express.json({ limit: '64kb' }));
app.use('/api/v1', createHealthRouter());
app.use('/api/v1/auth', authRouter(services));
app.use('/api/v1/inventario', inventoryRouter(services));
app.use('/api/v1/catalogos', catalogRouter(services));
app.use('/api/v1/compras/proveedores', supplierRouter(services));
app.use('/api/v1/compras/facturas/:id/adjuntos', attachmentRouter(services));
app.use('/api/v1/compras/facturas', invoiceRouter(services));
app.use('/api/v1/reportes', reportRouter(services));
app.use(notFound);
app.use(errorHandler(logger, services.audit));
return app;
}
```

Figura 9. app.ts · composición de la cadena

```ts
export function authenticate(verifier: AccessTokenVerifier): RequestHandler {
  return async (req, res, next) => {
    const header = req.headers.authorization;
    const match = header?.match(/^Bearer ([A-Za-z0-9._~+\/]+=*)$/i);
    if (!match?.[1] || match[1].length > 16_384) {
      throw new AppError(401, 'TOKEN_REQUIRED', 'Se requiere un access token Bearer.');
    }
    res.locals.identity = await verifier(match[1]);
    next();
  };
}

export function resolveUser(profiles: UserProfileRepository): RequestHandler {
  return async (_req, res, next) => {
    const identity = res.locals.identity as VerifiedIdentity;
    const user = await profiles.buscarIdentidad(identity.tenantId, identity.objectId);
    // Se usa únicamente identidad verificada, nunca correo o ID enviado por el body.
    if (!user) throw new AppError(403, 'USER_NOT_REGISTERED', 'Su perfil no está registrado.');
    res.locals.actor = user;
    if (!user.activo) throw new AppError(403, 'USER_DISABLED', 'Su perfil está inactivo.');
    const requiredRole = user.rol === 'ADMINISTRADOR' ? 'Administrador' : 'Mecanico';
    if (!identity.roles.includes(requiredRole)) {
      throw new AppError(403, 'ROLE_MISMATCH', 'El rol asignado no coincide con el perfil habilitado.');
    }
    next();
  };
}

export function requireRole(...roles: Array<'ADMINISTRADOR' | 'MECANICO'>): RequestHandler {
  return (_req, res, next) => {
    if (!res.locals.actor || !roles.includes(res.locals.actor.rol)) {
      throw new AppError(403, 'ROLE_FORBIDDEN', 'Su rol no permite esta operación.');
    }
    next();
  };
}
```

Figura 10. authentication.ts · eslabones de seguridad

Problema

Aplicar reglas HTTP en orden y detener la petición en cuanto una falla.

Solución

Cada middleware llama a next() o lanza un error. Cadena: requestContext → helmet → cors → JSON 64 KB → authenticate → resolveUser → requireRole → Zod → controlador → errorHandler.

Evidencia

Origen no permitido → 403; token inválido no llega al repositorio; entrada del mecánico → 403 ROLE_FORBIDDEN.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

6. Matriz de trazabilidad

Patrón	Requerimiento	Código	Prueba	Estado
Capas / monolito modular	RNF-07, RNF-12	modules/, infrastructure/	Casos de uso sin interfaz	Implementado
Repository + Unit of Work	RNF-04, RNF-07	contracts.ts, sql-*.ts	Integración SQL (base efímera)	Implementado
Transactional Outbox	RF-13, RNF-11	sql-alerts.ts, notification-worker.ts	Reservas y reintentos (18)	Implementado; correo pendiente
Singleton	RNF-09	sql-pool.ts	Misma instancia y reintento	Implementado
Facade	RF-08, RF-09, RNF-04	inventario.facade.ts	Concurrencia y rollback	Implementado
Observer	RF-12	transactional-publisher.ts	Umbral y no duplicados	Implementado
State	RF-12, RF-13	alerta.ts	Transiciones	Implementado
Strategy	RF-11	report-strategies.ts	PDF y Excel	Implementado
Chain of Responsibility	RF-02, RF-03, RNF-01	app.ts, authentication.ts	Rechazo 401 / 403	Implementado

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

7. Modelo C4: nivel 1 – Contexto

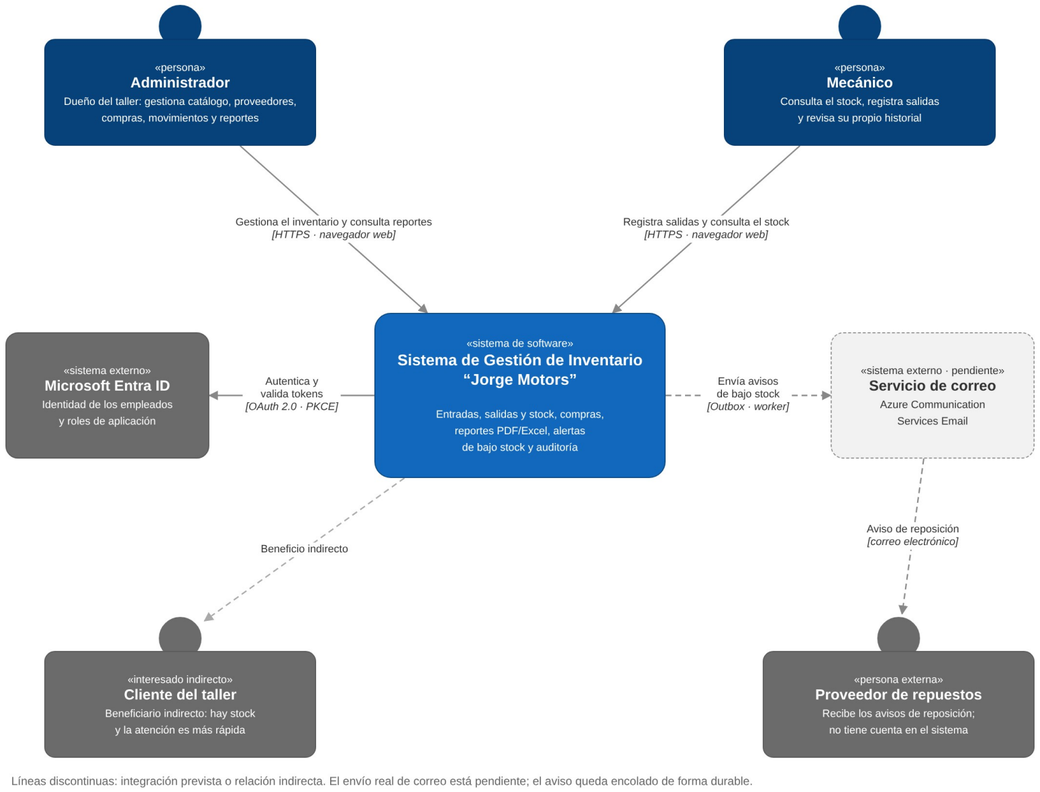


Figura 11. Vista de contexto (C4 – Nivel 1)

Elementos

- **Personas:** Administrador y Mecánico.
- **Externos:** Microsoft Entra ID y servicio de correo (pendiente).
- **Proveedor:** recibe avisos; no tiene cuenta.
- **Cliente del taller:** beneficio indirecto.

Decisiones que muestra

- La identidad se delega a Entra ID: el sistema no administra contraseñas.
- La relación con el proveedor es de salida: el aviso queda encolado de forma durable.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

7. Modelo C4: nivel 2 – Contenedores

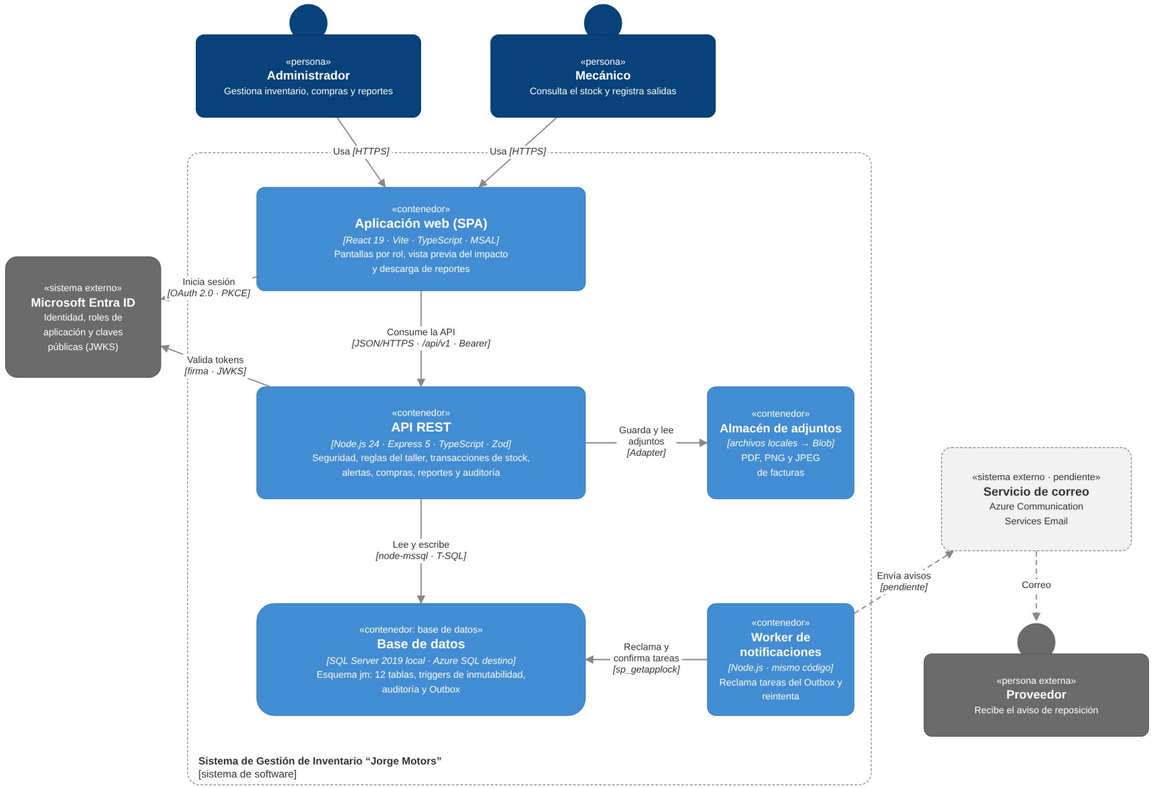


Figura 12. Vista de contenedores (C4 – Nivel 2)

Contenedor	Tecnología
Aplicación web	React 19 + Vite + MSAL
API REST	Node.js 24 + Express 5 + Zod
Base de datos	SQL Server / Azure SQL
Adjuntos	Archivos locales → Blob
Worker	Node.js, mismo código

Decisiones que muestra

- Solo la API accede a la base de datos.
- El envío de avisos está separado en el worker: si falla el correo, el movimiento no se revierte.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

7. Modelo C4: nivel 3 – Componentes

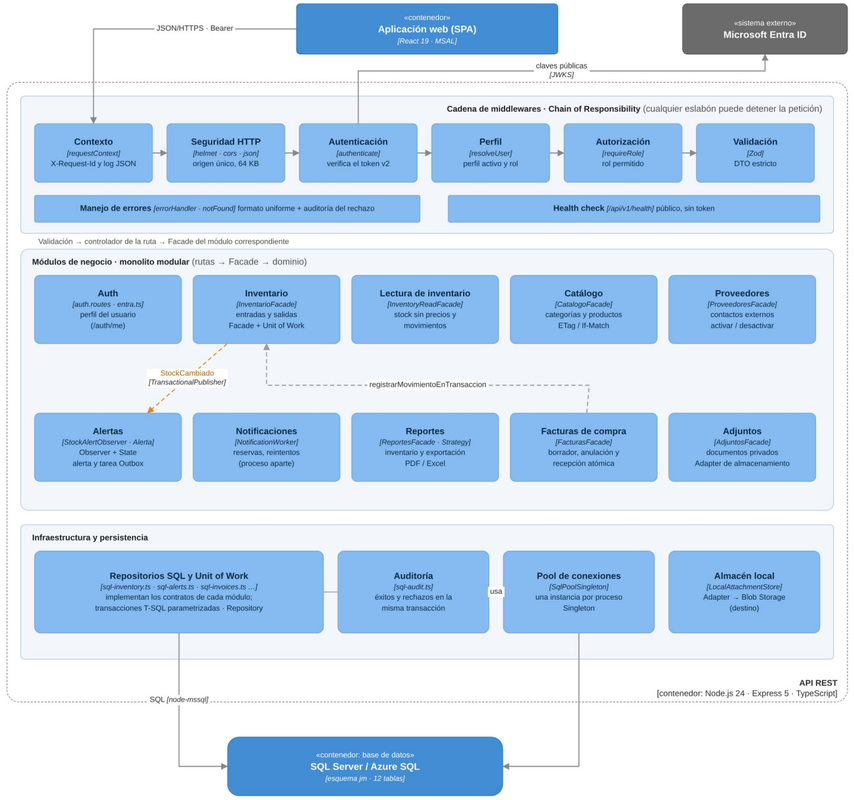


Figura 13. Componentes de la API REST (C4 – Nivel 3)

Cadena de middlewares

Contexto → seguridad HTTP → authenticate → resolveUser → requireRole → Zod → controlador → errorHandler.

Módulos de negocio

Auth, Inventario, Lectura, Catálogo, Proveedores, Alertas, Notificaciones, Reportes, Facturas y Adjuntos.

Persistencia

Repositorios SQL + Unit of Work, auditoría y SqlPoolSingleton.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

7. Modelo C4: nivel 4 – Código

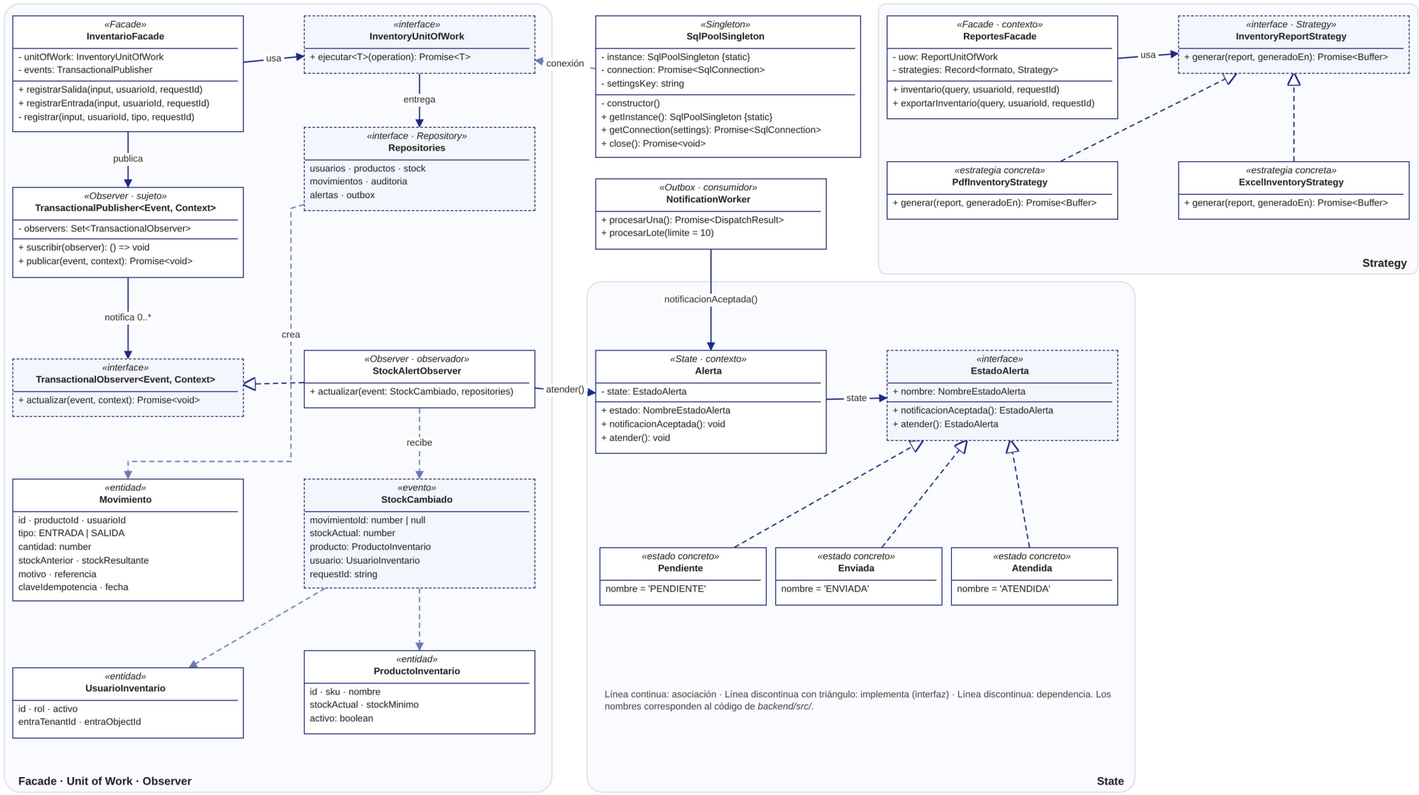


Figura 14. Diagrama de clases de los patrones (C4 – Nivel 4)

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

7. Vista de despliegue en Azure

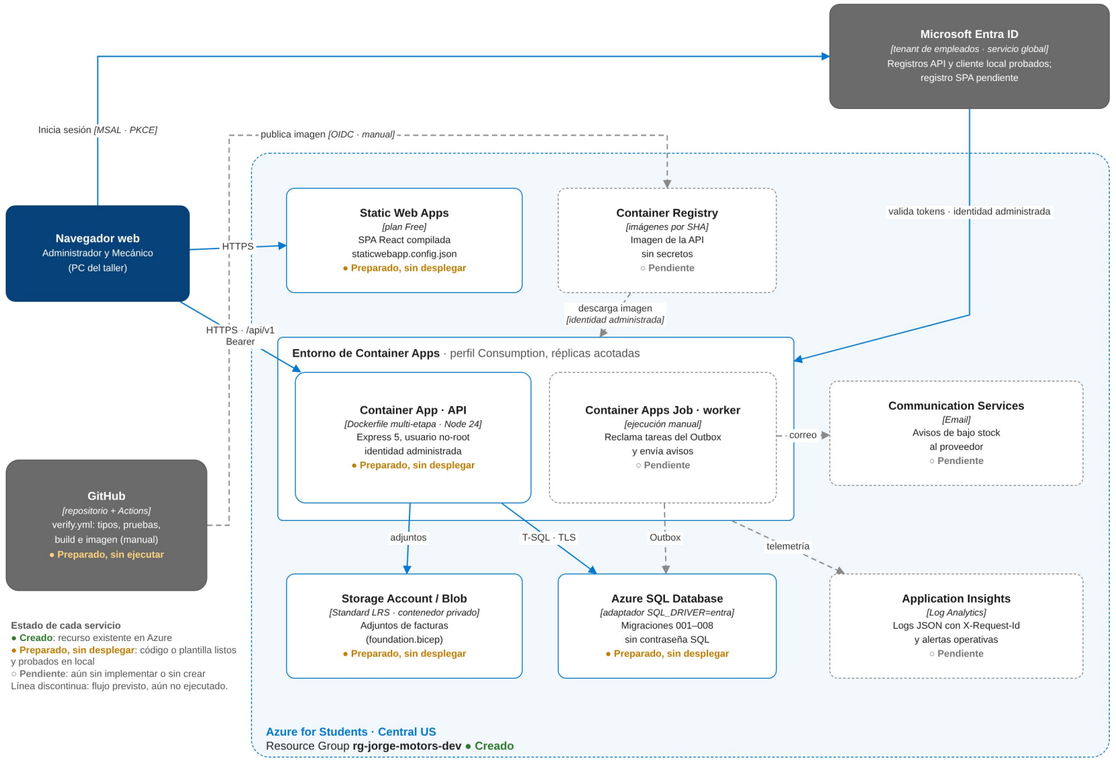


Figura 15. Vista de despliegue objetivo en Azure

1	5	4
creado	preparados	pendientes

Entrega de software

- Pull Request: tipos, pruebas y compilación.
- Imagen Docker sin secretos, etiquetada con el SHA.
- Despliegue iniciado manualmente.
- Migraciones con identidad distinta a la de ejecución.
- GitHub Actions con OIDC; acceso a SQL con identidad administrada.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

7. Decisiones arquitectónicas (ADR)

ADR	Decisión	Alternativa descartada	Consecuencia
01	Monolito modular en capas	Microservicios	Un despliegue; módulos con dependencias claras
02	node-mssql con T-SQL explícito	Prisma con SQLite / PostgreSQL	Control de bloqueos y transacciones; más SQL que mantener
03	Identidad con Microsoft Entra ID	Login propio con contraseñas y JWT	Sin contraseñas; roles, MFA y PKCE
04	Transactional Outbox para avisos	Enviar el correo en la petición	Ningún aviso se pierde; no hay entrega exactamente una vez
05	Adjuntos privados tras un Adapter	Archivos públicos	Local en el demo; Blob privado como destino
06	Despliegue manual en Azure	Despliegue automático	Control de costos y aprendizaje del equipo

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

8. Base de datos: migraciones y modelo conceptual

N.º	Contenido
001	usuario, categoria, producto, stock, movimiento
002	auditoria inmutable
003	alerta y notificacion_outbox
004	reservas e intentos del worker
005	ROWVERSION y alertas de catálogo
006	proveedor
007	factura_compra y sus líneas
008	factura_adjunto

Esquema **jm** · registro de versiones en dbo.jm_schema_migrations · comando pnpm db:migrate

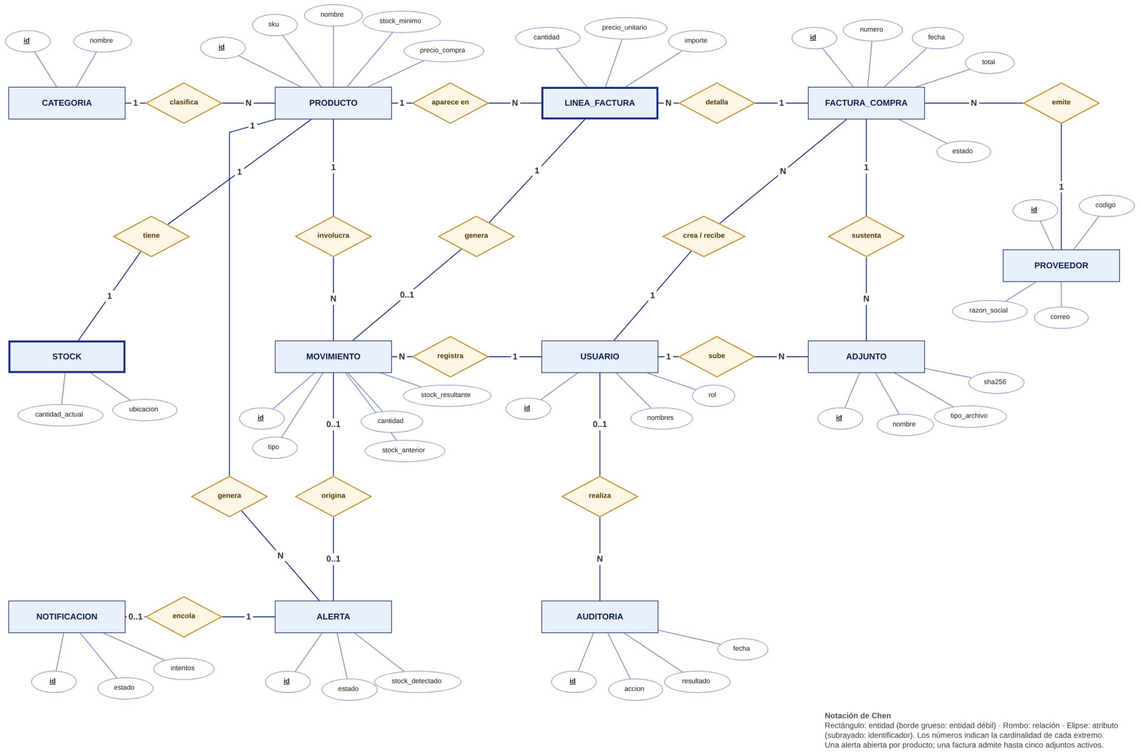


Figura 16. Modelo conceptual (notación de Chen)

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

8. Base de datos: modelo lógico

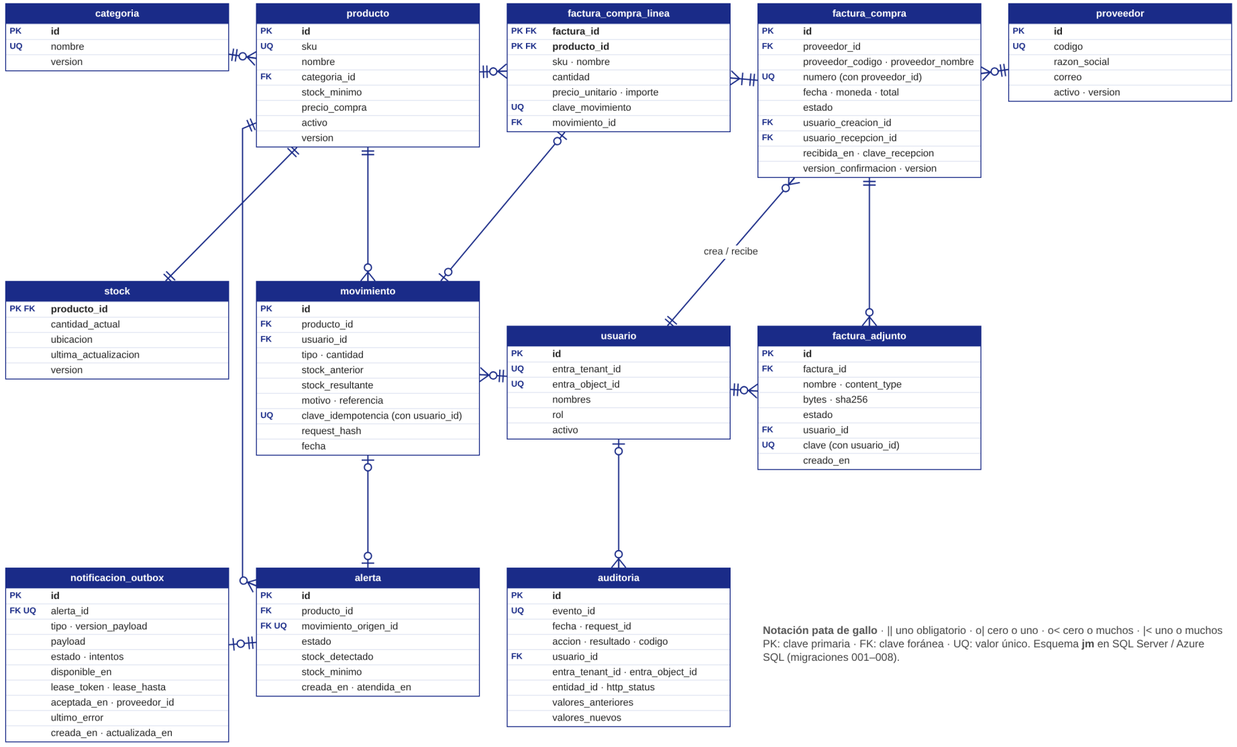


Figura 17. Modelo lógico relacional

Claves

- **PK** compuesta en factura_compra_linea.
- **1:1** producto–stock con producto_id como PK.
- **UQ** filtrado: un movimiento origina una sola alerta.
- **Sin borrado:** todas las FK en NO ACTION; se usa activo.
- **3FN** con redundancias intencionales para la trazabilidad.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

8. Base de datos: modelo físico

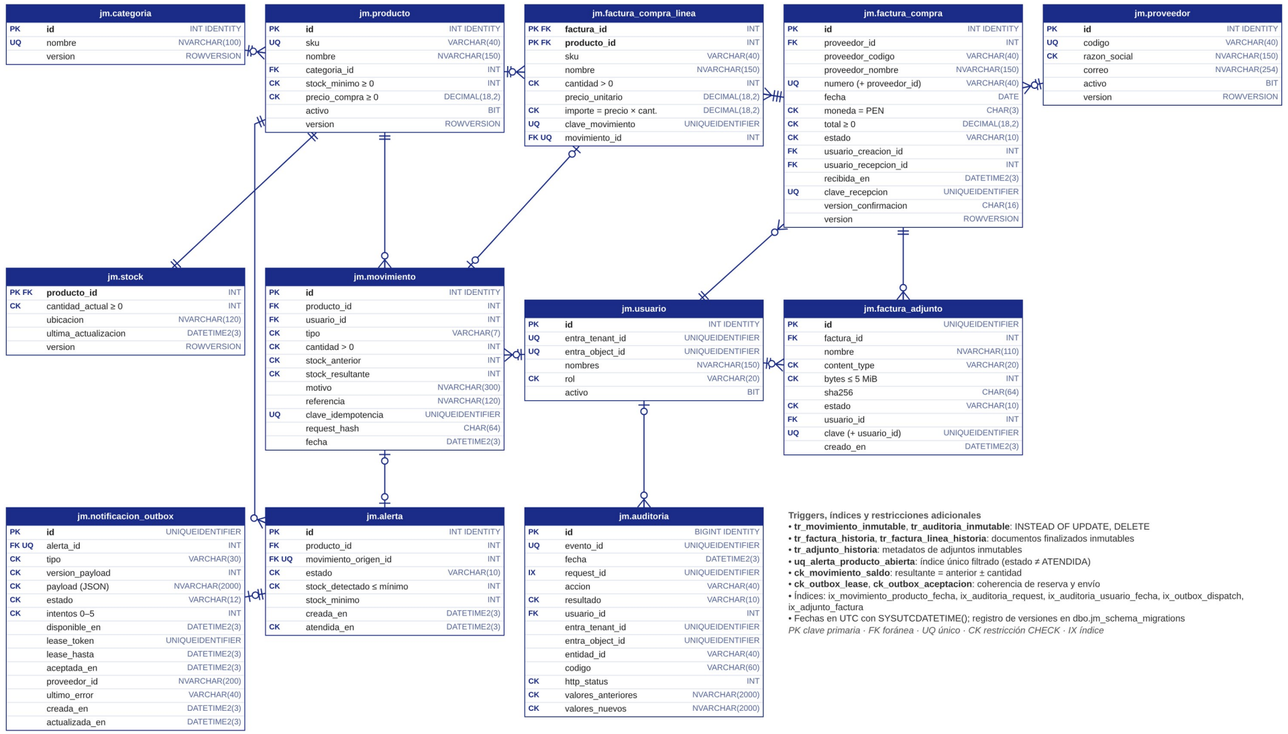


Figura 18. Modelo físico en SQL Server (esquema jm)

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

8. Consistencia del stock y concurrencia

1. **Transacción** — valida DTO y perfil activo
2. **Idempotencia** — bloquea la clave del usuario
3. **Bloqueo** — lee stock con UPDLOCK + HOLDLOCK
4. **Saldo** — reglas puras del dominio
5. **Movimiento** — actualiza stock y guarda historial
6. **Observer + commit** — alerta y Outbox, luego commit

Ejemplo: dos salidas a la vez

- Stock 3; dos mecánicos piden 2 unidades al mismo tiempo.
- El bloqueo por producto los atiende en orden.
- Se confirma una salida (saldo 1) y la otra se rechaza por stock insuficiente.
- Doble clic con la misma clave: un único movimiento.

Prueba de integridad	Resultado
UPDATE / DELETE en movimiento	Rechazado por trigger
Segunda alerta abierta	Rechazada por índice filtrado
Versión antigua (If-Match)	412 STALE_VERSION
Fallo del Outbox	Se revierte el movimiento
Recepción repetida	Stock sumado una vez

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

9. Demostración: navegación por rol

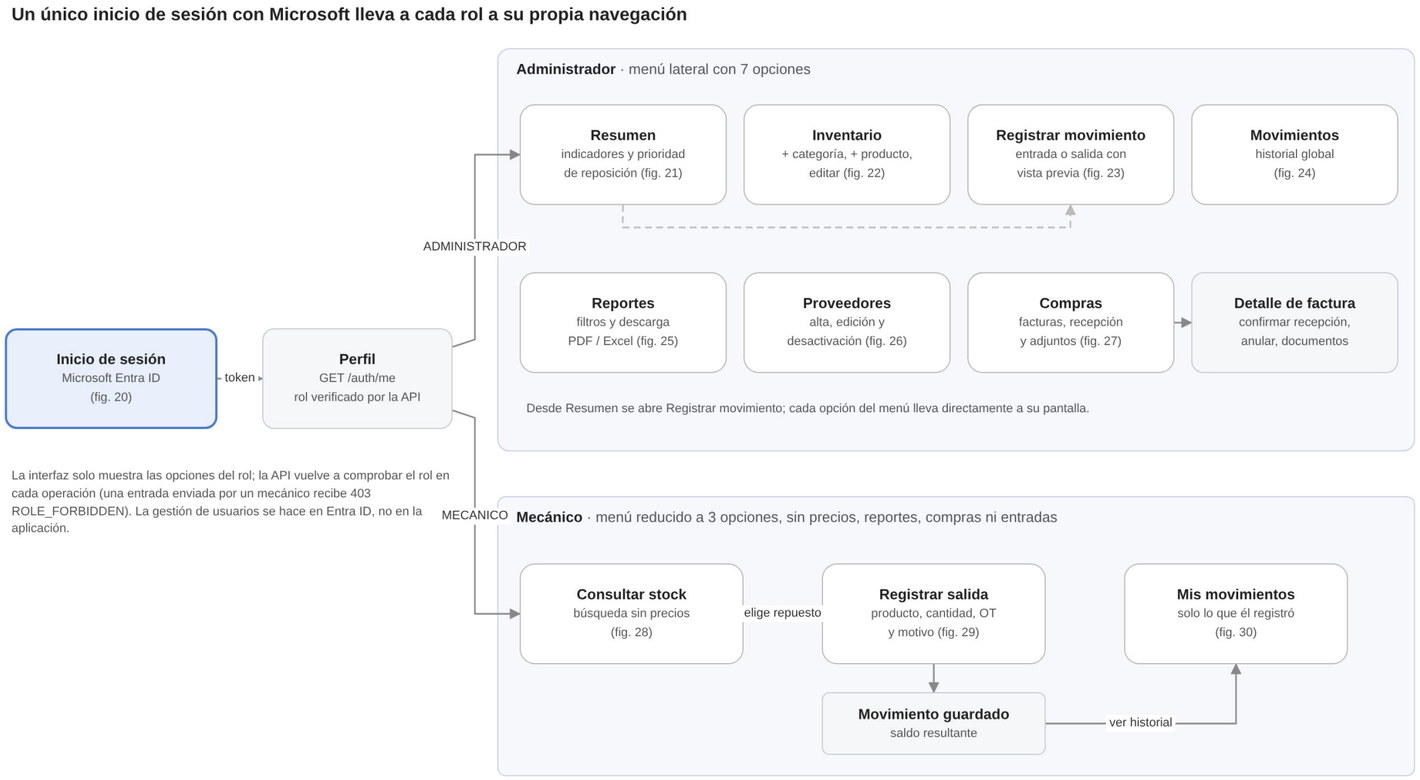


Figura 19. Mapa de navegación por rol

Administrador

Resumen, Inventario, Registrar movimiento, Movimientos, Reportes, Proveedores y Compras.

Mecánico

Consultar stock, Registrar salida y Mis movimientos.

Una entrada enviada por un mecánico recibe 403.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

9. Demostración: acceso y resumen

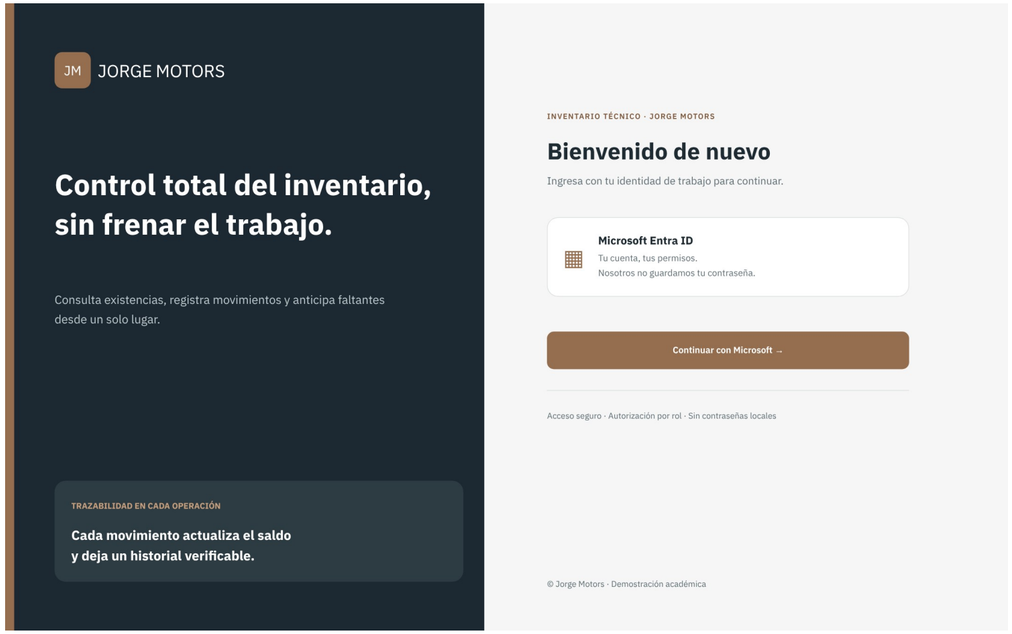


Figura 20. Inicio de sesión

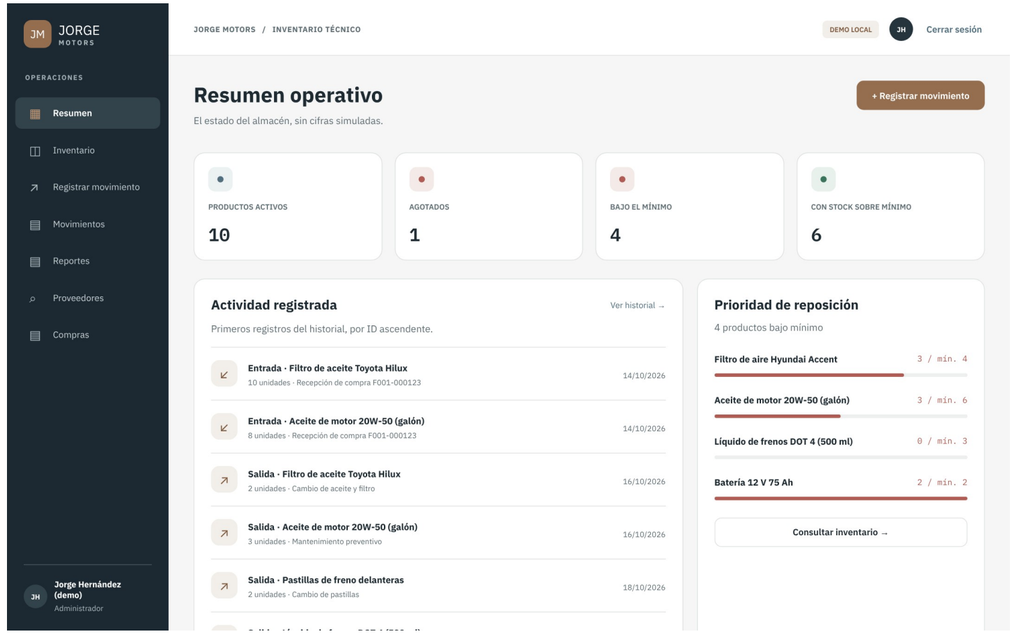


Figura 21. Resumen operativo

RF-01: ingreso con la cuenta de trabajo; sin contraseñas locales.

RF-11 / RF-12: indicadores y prioridad de reposición.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

9. Demostración: inventario y movimientos

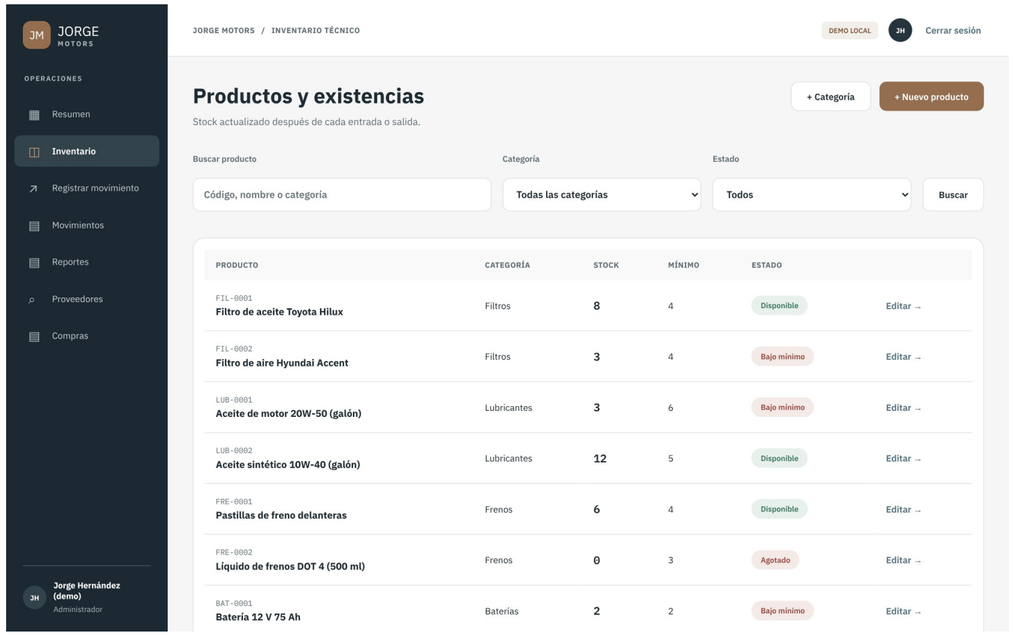


Figura 22. Productos y existencias

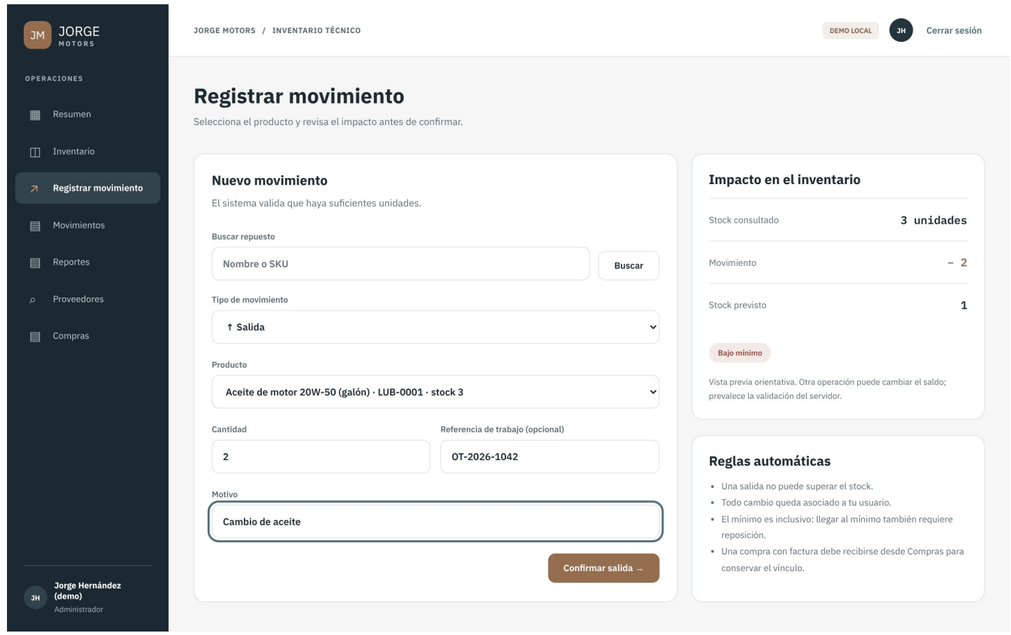


Figura 23. Registrar movimiento (Facade)

RF-05 / RF-06: catálogo con filtros y edición versionada.

RF-08 / RF-09: vista previa del stock y una sola confirmación.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

9. Demostración: historial y reportes

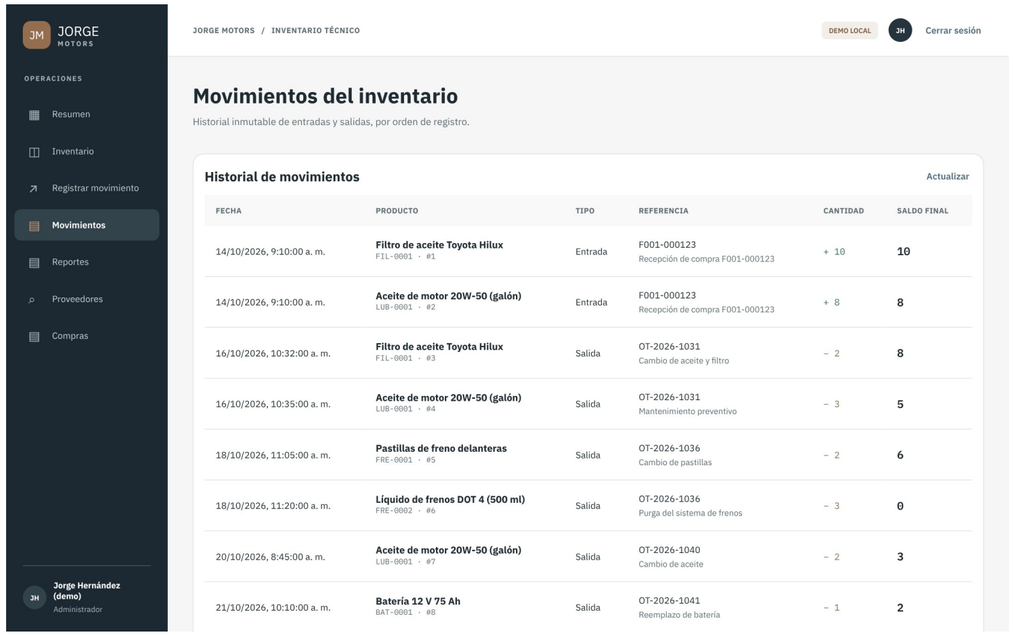


Figura 24. Historial de movimientos

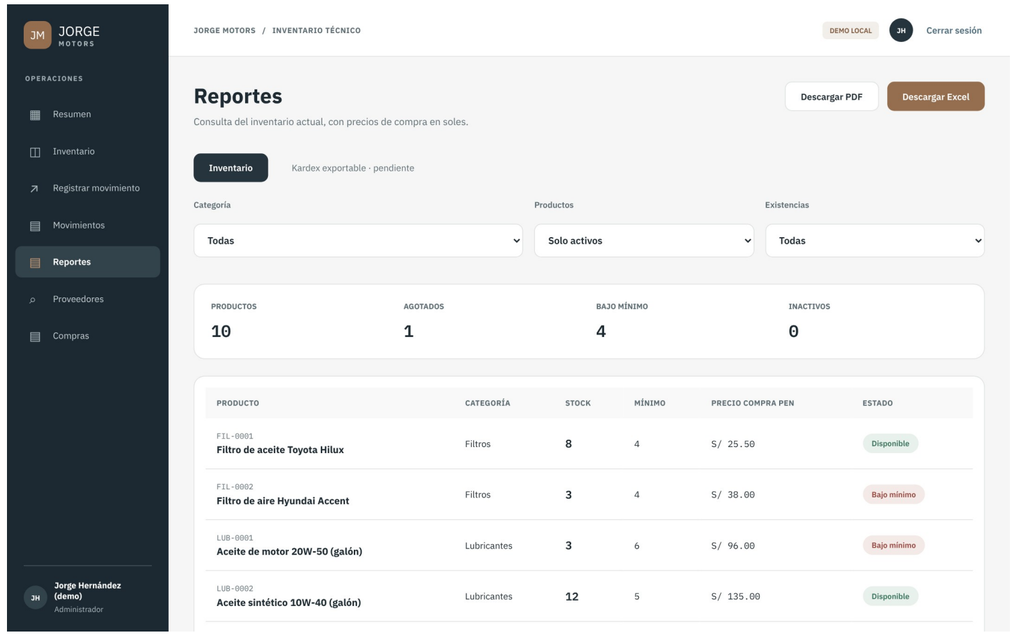


Figura 25. Reportes PDF / Excel (Strategy)

RF-10: historial inmutable; errores con movimiento compensatorio.

RF-11: filtros y exportación en dos formatos.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

9. Demostración: proveedores y compras

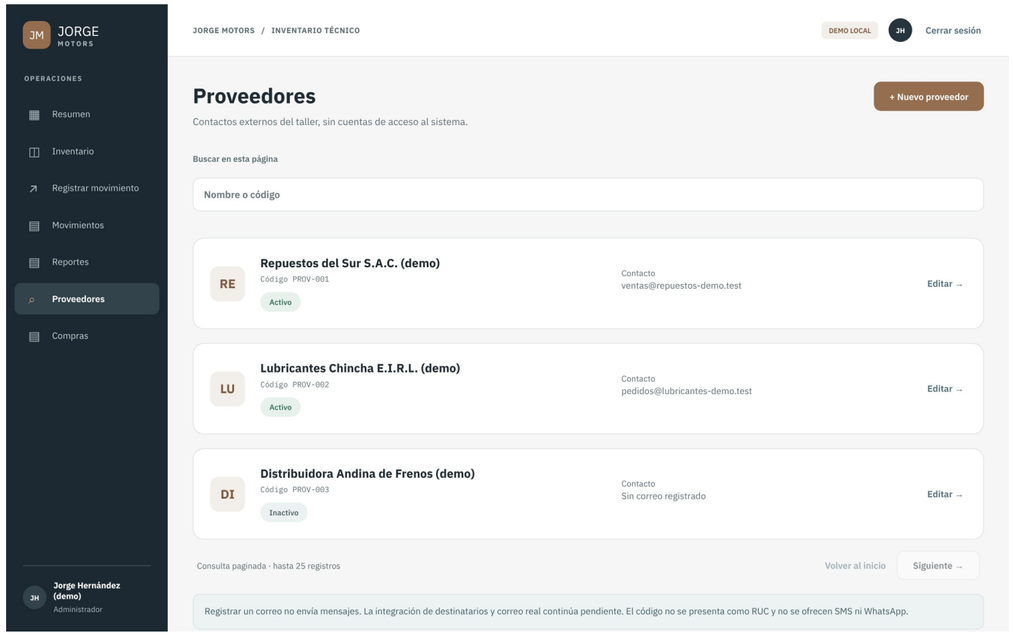


Figura 26. Proveedores

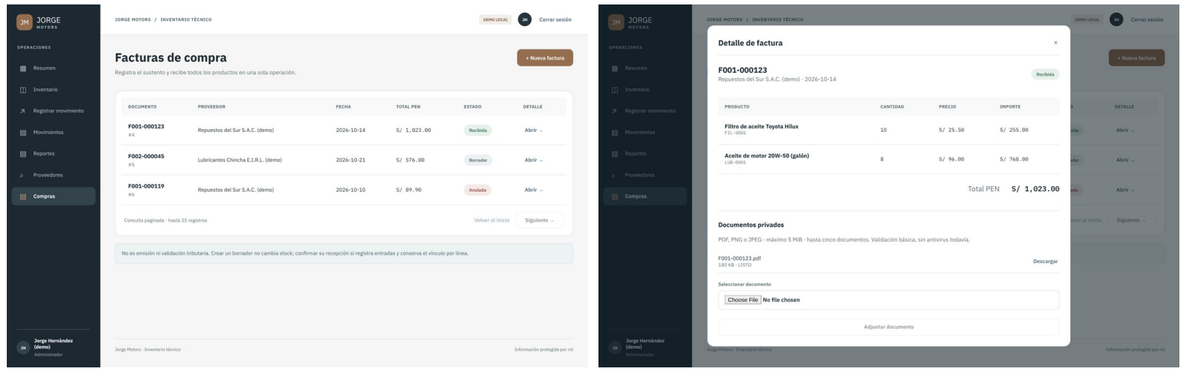


Figura 27. Facturas de compra y adjuntos

RF-07: alta, edición y desactivación sin borrar historia.

RF-14 / RF-15: recepción atómica y documentos privados.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

9. Demostración: vista del mecánico

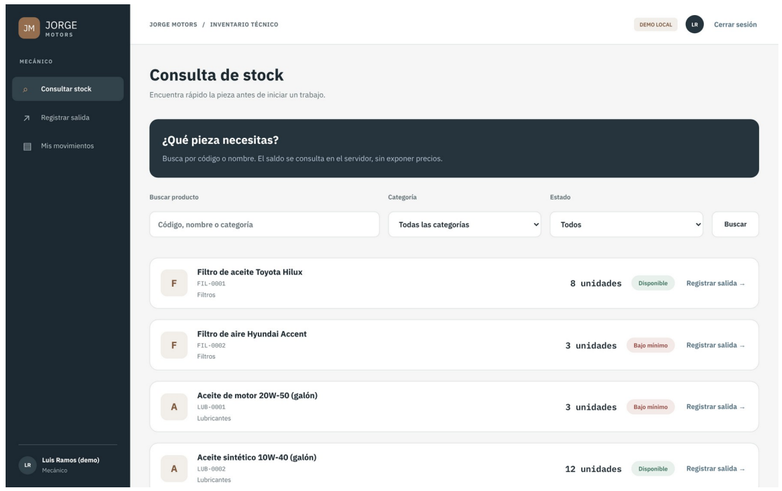


Figura 28. Consultar stock

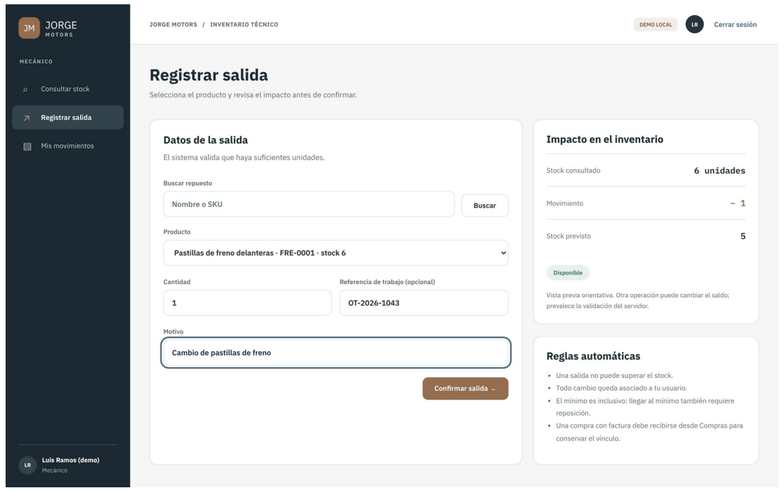


Figura 29. Registrar salida

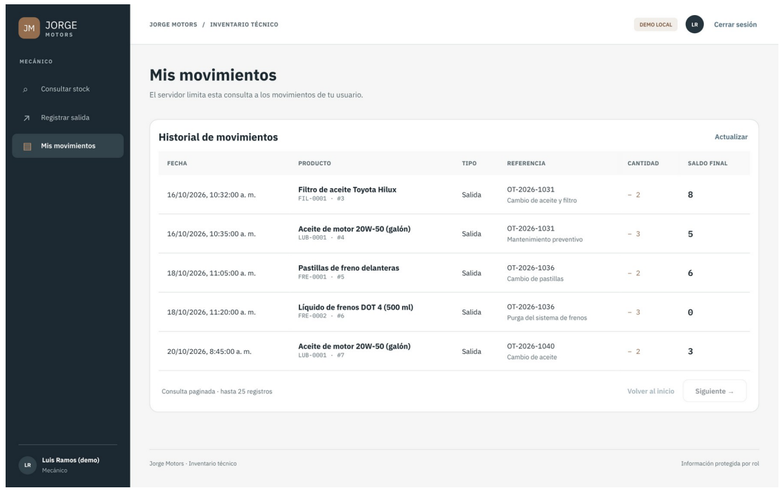


Figura 30. Mis movimientos

- **RF-02 / RF-03:** menú de tres opciones; sin precios, compras ni reportes.
- **RNF-13:** la salida se registra en un solo formulario con vista previa del stock.
- **RF-10:** el historial se filtra en el servidor por la identidad del mecánico.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

10. Resultados: pruebas automatizadas

98 unitarias y HTTP del backend	119 integración con SQL Server
6 unitarias del frontend	9 escenarios de navegador

232 pruebas aprobadas

Terminal · pruebas automatizadas de Jorge Motors

```
jorge-motors/backend $ pnpm exec tsc --noEmit -p tsconfig.json
✔ Chequeo de tipos sin errores
jorge-motors/backend $ pnpm exec tsx --test --test-reporter=spec tests/*.test.ts
✔ umbral inclusivo: mínimo y cero requieren alerta; por encima no (0.925655ms)
✔ State: pendiente puede notificarse y luego atenderse (0.214945ms)
✔ State: reponer antes de enviar atiende la alerta sin afirmar envío (0.099768ms)
✔ State: reconstruye estados persistidos y rechaza valores desconocidos (0.19911ms)
✔ Observer: suscripción, orden esperado y baja sin notificar dos veces al mismo objeto (0.761687ms)
✔ Observer: un fallo se propaga a la UoW y no continúa silenciosamente (0.433409ms)
✔ Adjuntos: firma, MIME y extensión coinciden; SHA-256 y tamaño exactos (1.457368ms)
✔ Adjuntos: rechaza rutas, doble extensión, controles, HTML y contenido incompatible (1.129705ms)
✔ Adjuntos: no admite vacío ni tamaño mayor a 5 MiB (0.314302ms)
✔ Adjuntos: datos inválidos se rechazan antes de SQL o disco (0.98062ms)
  … 88 pruebas más (todas ✔) …
ℹ tests 98
ℹ pass 98
ℹ fail 0
ℹ duration_ms 6415.281107

jorge-motors/frontend $ pnpm exec tsx --test --test-reporter=spec tests/*.test.ts
✔ API: solo HTTPS o loopback explícito y sin credenciales, query ni fragmento (1.274983ms)
✔ API: token solo en cabecera, sin cookies, caché, redirects ni token en URL (20.12237ms)
✔ API: error mantiene código HTTP/correlación y no convierte fallo en lista vacía (0.687631ms)
✔ API: no enlaza fetch al receptor ApiClient (compatibilidad navegador) (0.436981ms)
✔ Dinero exacto y estados inclusivos, sin convertir a Number (0.264305ms)
✔ Idempotencia: la misma intención conserva clave tras reintento, otra cambia y éxito reinicia (0.226793ms)
ℹ tests 6
ℹ pass 6
ℹ fail 0
ℹ duration_ms 270.257582
```

Figura 31. Ejecución de las pruebas unitarias

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

10. Resultados: evidencia con identidades reales

Operación	Usuario	Resultado
Consultar perfil (/auth/me)	Administrador	200, perfil 1
Entrada de 5 unidades	Administrador	201; stock 0 → 5
Salida de 2 unidades	Mecánico	201; stock 5 → 3
Registrar una entrada	Mecánico	403 ROLE_FORBIDDEN; stock sin cambios
Salida 3 → 2 (mínimo 2)	Mecánico	201; alerta y aviso PENDIENTE
Entrada 2 → 4	Administrador	201; alerta ATENDIDA, aviso CANCELADO
Editar con versión antigua	Administrador	412 STALE_VERSION
Exportar PDF y Excel	Administrador	200; archivos generados y auditados

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

10. Resultados: metas de calidad

Atributo	Meta	Resultado	Estado
Seguridad	100 % de accesos no autorizados denegados	401 / 403 con identidades reales y de prueba	Cumplida
Integridad	0 inconsistencias de stock simultáneas	Sin negativos, dobles descuentos ni movimientos parciales	Cumplida
Auditoría	Autor, fecha y saldos en cada movimiento	Historial y auditoría inmutables por trigger	Cumplida
Fiabilidad	Ningún aviso perdido	Outbox con reservas y hasta 5 intentos	Cumplida en simulación
Mantenibilidad	Casos de uso probados sin interfaz	98 pruebas unitarias / HTTP	Cumplida
Usabilidad	Salida en pocos pasos	Un formulario con vista previa; axe sin infracciones A/AA	Cumplida
Rendimiento	Búsqueda ≤ 2 s (p95)	Consultas paginadas e indexadas	Pendiente de medir
Recuperación	Restauración de prueba	Depende de Azure SQL	Pendiente

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

10. Pendientes y trabajo futuro

Correo real

Adaptador de Azure Communication Services, destinatarios y Job programado.

Despliegue en Azure

Construir la imagen, ejecutar el workflow y crear Azure SQL.

Registro SPA en Entra ID

Inicio de sesión real desde el navegador.

Blob Storage y antivirus

Almacenamiento definitivo de los adjuntos.

Funciones futuras

Kardex exportable, valorización, tablero de alertas y gestión web de usuarios.

Medición de rendimiento

Verificar la meta de 2 s con datos y concurrencia definidos.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

11. Conclusiones

1

Del problema al sistema

16 RF y 16 RNF; 15 funcionales implementados y verificados.

2

Arquitectura que escala

Capas + monolito modular: compras y reportes sin tocar los movimientos.

3

Patrones en el código

Los seis GoF implementados y probados, no solo documentados.

4

Stock consistente

Transacción, bloqueo e idempotencia: sin negativos ni duplicados.

5

Seguro y fiable

Entra ID sin contraseñas y Outbox sin avisos perdidos.

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados

12. Bibliografía y herramientas

Referencias

- Bass, L., Clements, P., y Kazman, R. (2021). Software architecture in practice (4.ª ed.). Addison-Wesley.
- Brown, S. (2018). The C4 model for software architecture. https://c4model.com
- Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1995). Design patterns. Addison-Wesley.
- Microsoft. (s. f.). Transactional outbox pattern; Microsoft Entra authentication with Azure SQL. Microsoft Learn.
- Sakimura, N., Bradley, J., y Agarwal, N. (2015). Proof Key for Code Exchange (RFC 7636). IETF.
- Antecedentes: Malquichagua Chu (2022); Adriazola y Kjuro (2024); Galarza Arias (2021); Cevallos Suárez (2026).

Herramientas

React + Vite	TypeScript
Node.js + Express	node-mssql
SQL Server / SSMS	Microsoft Entra ID
Zod	PDFKit + ExcelJS
Playwright + axe	Docker + GitHub Actions
draw.io	Figma

Universidad Nacional de Cañete – UNDC 2026. Todos los derechos reservados